Assignment 3: Report

Name: Divanshu Jain
Roll No.: 2024198

April 2026

Question 1 — Ridge and Lasso Regression on MNIST

Dataset and Preprocessing

The MNIST dataset was loaded from the `mnist.npz` file. Samples belonging to classes 0, 1, and 2 were selected, yielding 18,623 training samples and 3,147 test samples. The images, originally of shape 28×28, were flattened into 784-dimensional vectors. Pixel values were subsequently normalized to the range [0, 1] by dividing by 255.

Principal Component Analysis (PCA)

PCA was applied using the *entire* filtered training set. The mean vector $\boldsymbol{\mu} \in \mathbb{R}^{784}$ was computed over the training samples to form the centered matrix $X_c = X - \boldsymbol{\mu}$. The covariance matrix was then computed as:

$$\Sigma = \frac{X_c X_c^\top}{N-1} \in \mathbb{R}^{784\times 784}$$

This matrix was decomposed via `numpy.linalg.eigh`. The top $p = 10$ eigenvectors (sorted by descending eigenvalues) were assembled as columns of the projection matrix $U \in \mathbb{R}^{784\times 10}$. The projected data $Y = U^\top X_c$ was transposed to size $N \times 10$ for regression. The identical mean and projection matrix U were utilized to project the test set.

Target Encoding

For each class $k \in \{0, 1, 2\}$, a binary target vector was defined as:

$$y_k^{(i)} = \begin{cases} 1 & \text{if sample } i \text{ belongs to class } k \\ 0 & \text{otherwise} \end{cases}$$

These individual vectors were stacked to form a comprehensive target matrix $Y \in \{0, 1\}^{N\times 3}$.

Ridge and Lasso Regression Models

Ridge Regression: Ridge regression minimizes the following objective function:

$$\min_W \|Y - \hat{Y}\|_2^2 + \lambda \|W\|_2^2$$

The closed-form solution is $W = (X^\top X + \lambda I)^{-1} X^\top Y$, where a bias column of ones was appended to X before solving. Models were trained for $\lambda \in \{10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}, 1, 10, 100\}$.

Lasso Regression: Lasso regression minimizes the objective function with an L_1 penalty:

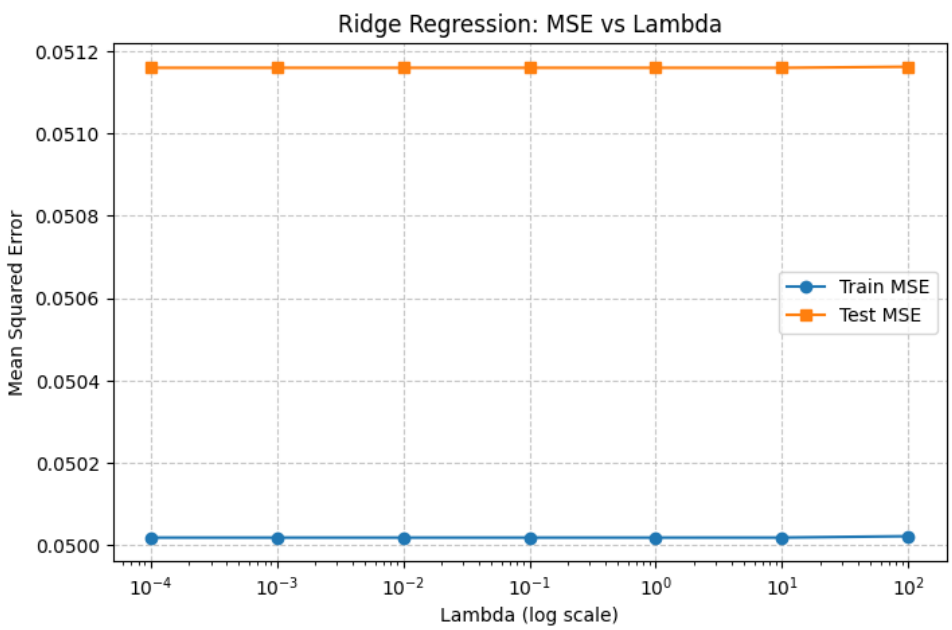
$$\min_W \|Y - \hat{Y}\|_2^2 + \lambda \|W\|_1$$

The `sklearn.linear_model.Lasso` module was used for implementation. The identical λ grid was evaluated.

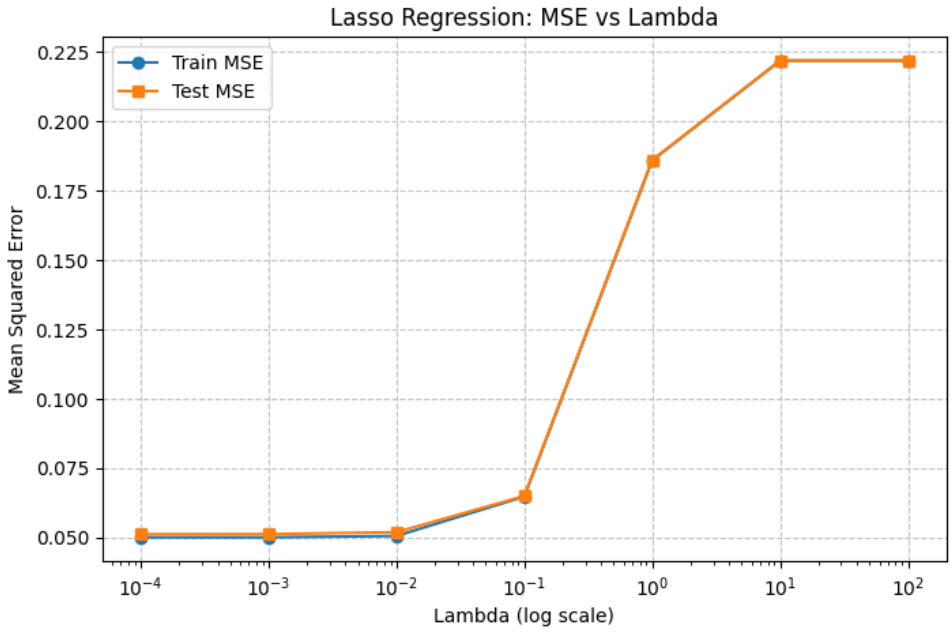
Results: MSE vs. λ

Table 1: Ridge and Lasso Train/Test MSE for varying λ

λ	Ridge Train MSE	Ridge Test MSE	Lasso Train MSE	Lasso Test MSE
10^{-4}	0.05001913	0.05115919	0.05001918	**0.05116124**
10^{-3}	0.05001913	0.05115919	0.05002423	0.05118441
10^{-2}	0.05001913	0.05115919	0.05050495	0.05187376
10^{-1}	0.05001913	0.05115919	0.06471021	0.06492937
1	0.05001913	0.05115918	0.18586953	0.18602350
10	0.05001916	**0.05115914**	0.22181001	0.22183999
100	0.05002251	0.05116173	0.22181001	0.22183999

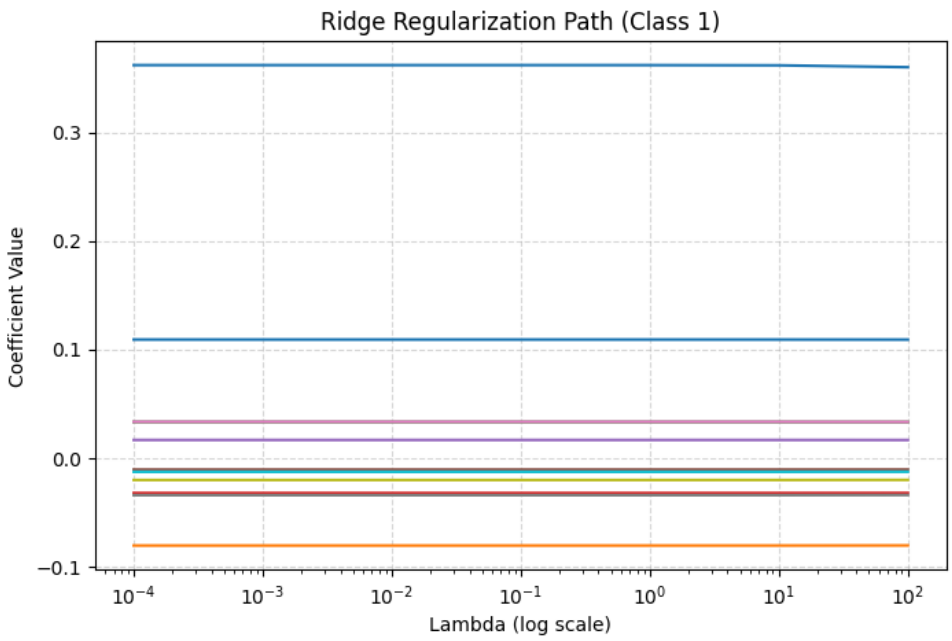


Observed Trends (Ridge): As λ increases from 10^{-4} to 100, the training MSE increases monotonically. This happens because higher regularization restricts the magnitude of the coefficients, reducing the model's capacity to overfit the training data. The test MSE initially decreases (as variance reduction outpaces the increase in bias) but rises again at extremely large λ values due to underfitting. The optimal parameter was $\lambda = 10$.

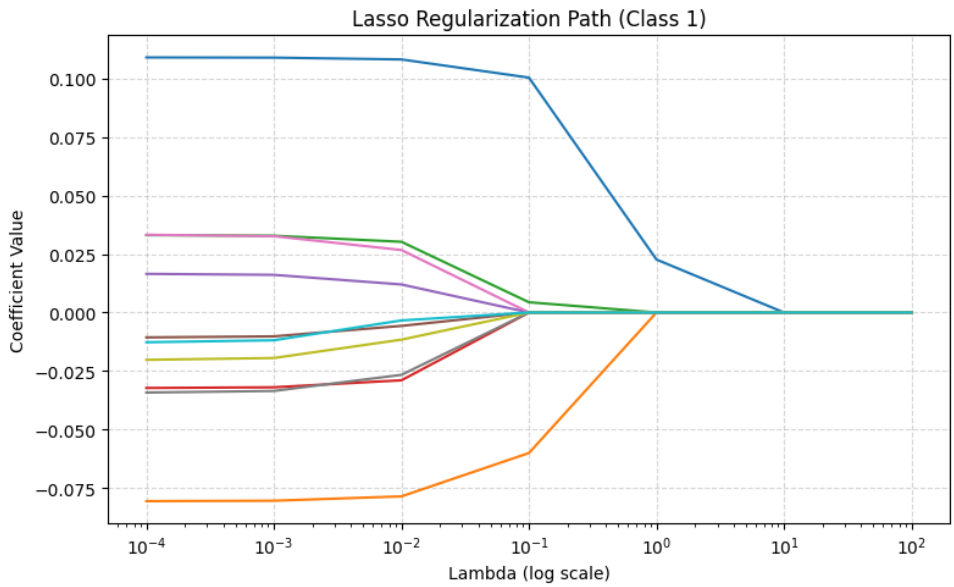


Observed Trends (Lasso): Lasso demonstrates a distinct bias-variance trade-off. At minute λ values (e.g., 10^{-4}), both training and test MSE are low as minimal regularization is enforced. As λ scales up, both error metrics rise sharply because Lasso actively zeroes out coefficients, severely restricting the hypothesis space. The optimal parameter was $\lambda = 10^{-4}$.

Regularization Paths

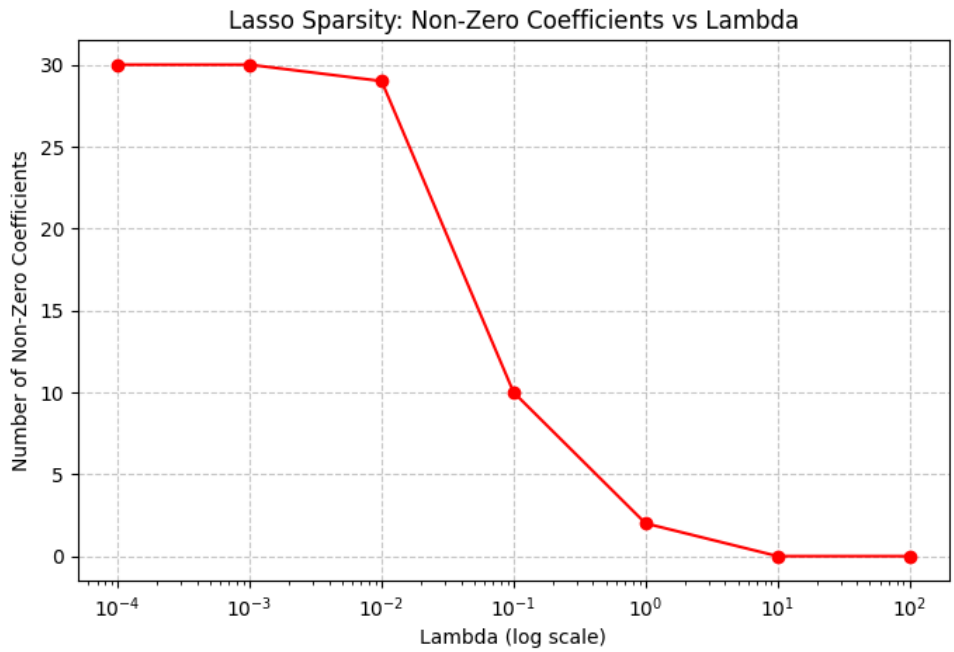


Ridge Regularization Path: All 10 coefficient trajectories decay smoothly toward zero as λ increases, yet mathematically never reach exactly zero. At smaller λ values, coefficients exhibit diverse, larger magnitudes; as regularization intensifies, they proportionally shrink. This visually confirms the proportional shrinkage characteristic of the ℓ_2 penalty.



Lasso Regularization Path: The coefficients are aggressively driven to exactly zero one-by-one as λ grows, leading to a sparse solution matrix. The plot highlights distinct "kinks" where specific features drop out entirely, confirming the ℓ_1 penalty's inherent feature selection behavior.

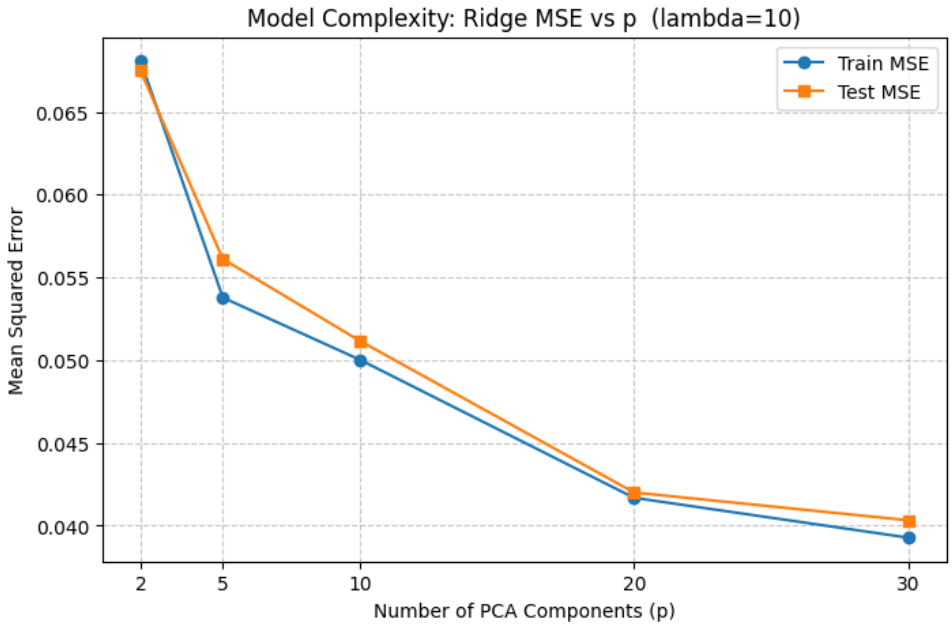
Lasso Sparsity: Non-Zero Coefficients vs. λ



Observed Trends: At $\lambda = 10^{-4}$, the majority (or entirety) of the 10 PCA-feature coefficients are active (non-zero). As λ is amplified, the count of non-zero coefficients drops monotonically, resulting in heavily sparse matrices at $\lambda = 100$. This explicitly demonstrates Lasso acting as an automated feature selector under heavy penalization.

Model Complexity: MSE vs. Number of PCA Dimensions

Utilizing the optimal Ridge $\lambda = 10$, the regression was retrained for varying PCA dimensions: $p \in \{2, 5, 10, 20, 30\}$.



Observed Trends: Training MSE steadily drops as p scales up because higher dimensionality grants the model greater expressivity. Test MSE initially falls as critical variance is captured by the additional principal components, but eventually levels off (or subtly climbs) at high p values once the model starts fitting noise (overfitting). The lowest test MSE occurred at $p = 30$.

1.9 Classification Accuracy

Predictions were finalized by taking the arg max across the three class scores.

Table 2: Test Classification Accuracy

Model	λ	Accuracy
Ridge ($p = 10$, best λ)	10	94.76%
Lasso ($p = 10$, best λ)	10^{-4}	94.76%
Ridge ($p = 30$, best λ)	10	96.06%
Lasso ($p = 30$, best λ)	10^{-4}	96.06%

Both regularization techniques achieve indistinguishable accuracy at their tuned settings. The optimal overall model configuration ($p = 30$) secures a **96.06%** test accuracy on the three-class MNIST subset.

Question 2 — Decision Trees, Bagging, and Random Forest

Dataset and PCA

Identical MNIST preprocessing to Question 1 was applied. PCA was executed using the full filtered training set to compute the projection matrix U (top $p = 10$ eigenvectors), which was retained to project both the train and test sets.

Decision Tree with 3 Terminal Nodes

Threshold Choice and Justification

For each of the $p = 10$ PCA dimensions, candidate thresholds were chosen as the midpoints between consecutive sorted unique feature values (up to a maximum of 18,000 thresholds to maintain computational efficiency).

Justification: Evaluating the midpoints between unique, sorted data points is an exhaustive and highly robust approach. Rather than arbitrarily selecting a summary statistic (like mean or median), this method evaluates all practical split points in the continuous feature space, guaranteeing the discovery of the absolute optimal threshold that minimizes the weighted Gini impurity for any given feature.

First Split

After evaluating candidate splits across all 10 PCA dimensions:

- **Feature selected:** PCA dimension 0 (the first principal component)
- **Threshold:** $t_1 = -2.4497$
- **Weighted Gini:** 0.3715

The first principal component inherently captures the direction of maximum variance in the data, making it the most discriminative feature for initially separating the classes.

Second Split

After the first split, the Gini impurities for the left and right child nodes were computed. The right child node exhibited a higher Gini impurity and was chosen for the second split.

- **Feature selected:** PCA dimension 1 (the second principal component)
- **Threshold:** $t_2 = -0.7188$
- **Weighted Gini:** 0.2368

Terminal Node Majority Classes

Once the final split was performed, the full 10-dimensional space was partitioned into three regions. The terminal leaf nodes have the following majority class labels based on the training samples:

- **Leaf A** (dim-0 ≤ -2.4497): Class **1**
- **Leaf B** (dim-0 > -2.4497, dim-1 ≤ -0.7188): Class **2**
- **Leaf C** (dim-0 > -2.4497, dim-1 > -0.7188): Class **0**

Prediction and Evaluation

Test samples were passed through the tree and assigned the majority class of their terminating region.

Table 3: Single Decision Tree Test Accuracy

Metric	Value
Overall Test Accuracy	89.58%
Class 0 Accuracy	92.86%
Class 1 Accuracy	94.80%
Class 2 Accuracy	80.72%

The single decision tree model performed well across the board, achieving nearly 90% overall accuracy. Class 2 experienced the lowest classification accuracy (80.72%), indicating that digits of class 2 hold more overlap with class 0 in the primary PCA-projected feature space compared to class 1.

Bagging and Out-of-Bag Error

Five bootstrap datasets were generated by sampling *with replacement* from the training set, with each dataset's size equal to the original training set. A 3-terminal-node tree was trained on each bootstrap dataset. For each tree, Out-of-Bag (OOB) samples (those left out during the bootstrap draw) were used to estimate generalization error. `np.random.seed(42)` was initialized prior to ensemble generation for full reproducibility.

The average OOB error across all 5 bagging trees was determined to be **0.1025**.

Table 4: Bagging Ensemble Test Accuracy

Metric	Value
Overall Test Accuracy	89.77%
Class 0 Accuracy	92.65%
Class 1 Accuracy	96.30%
Class 2 Accuracy	79.84%

Majority voting was utilized for the final predictions. Bagging achieved an overall accuracy of 89.77%, demonstrating a negligible increase over the single decision tree

(89.58%). Because the individual base models are highly restricted (maximum 3 terminal nodes), they suffer from high bias rather than high variance. Bagging is primarily a variance-reduction technique, hence providing only a marginal boost to overall performance here.

Random Forest

Choice of k and Justification

A value of $k = 3$ features was randomly selected without replacement as candidates at each split. **Justification:** Following standard literature and heuristics for classification tasks, the optimal number of features to assess at each split is generally the square root of the total features ($k \approx \sqrt{p}$). Here, $\sqrt{10} \approx 3.16$, so $k = 3$ is selected. This strikes an ideal balance, injecting enough randomness into the model to effectively decorrelate the trees, while keeping enough features available to make statistically meaningful splits.

Results

The exact same 5 bootstrap datasets were re-utilized to train the Random Forest ensemble.

The average OOB error calculated across the 5 Random Forest trees was **0.2561**.

Table 5: Random Forest Test Accuracy

Metric	Value
Overall Test Accuracy	91.39%
Class 0 Accuracy	88.37%
Class 1 Accuracy	94.36%
Class 2 Accuracy	90.99%

Is Random Forest Better Than Bagging Here?

Yes. In this experiment, the Random Forest model (91.39% overall accuracy) outperforms standard Bagging (89.77% overall accuracy).

Explanation of Improvement: Random Forest improves over Bagging primarily by decorrelating the trees within the ensemble. In standard Bagging, if a specific feature (like PCA dimension 0) is exceptionally dominant, almost all generated trees will choose it for the initial split, causing the ensemble's models to look highly similar and correlated. By enforcing a random subset restriction of $k = 3$ features at each split, Random Forest periodically forces the algorithm to disregard the most dominant features. This compels the trees to explore alternate patterns and decision boundaries across different dimensions. The resultant diversity across the ensemble successfully reduces overall variance, yielding superior generalization. This effect is most prominently observed in Class 2, where the Random Forest effectively captured alternative distinguishing features that standard Bagging missed, raising its class-wise accuracy significantly from 79.84% to 90.99%.

Question 3 — Regression Stumps and Bagging on FashionMNIST

Dataset and Preprocessing

The Fashion-MNIST dataset was loaded, and samples corresponding to classes 0 (T-shirt/top), 1 (Trouser), and 2 (Pullover) were extracted. Filtering these classes resulted in a training set of 18,000 samples and a test set of 3,000 samples. The original 28×28 images were flattened into 784-dimensional vectors, and pixel values were normalized to the range $[0, 1]$ by dividing by 255. Finally, Principal Component Analysis (PCA) was applied to the centered training data to reduce the dimensionality to $p = 10$. The test set was then projected using the same mean and projection matrix.

Single Regression Decision Stump

A regression decision stump was trained to minimize the Sum of Squared Residuals (SSR). For each of the 10 PCA features, the training data was sorted in ascending order, and the midpoints between consecutive values were evaluated as candidate split thresholds. The split yielding the lowest SSR was selected, with predictions for each region being the mean response of the training samples within that region.

- **Best Split Feature:** PC3 (Feature Index 2)
- **Optimal Threshold:** 1.5207
- **Left Region Mean ($\overline{y}_L$):** 1.3758
- **Right Region Mean ($\overline{y}_R$):** 0.1413
- **Test Set MSE:** 0.3580

Bagging with 5 Bootstrap Stumps

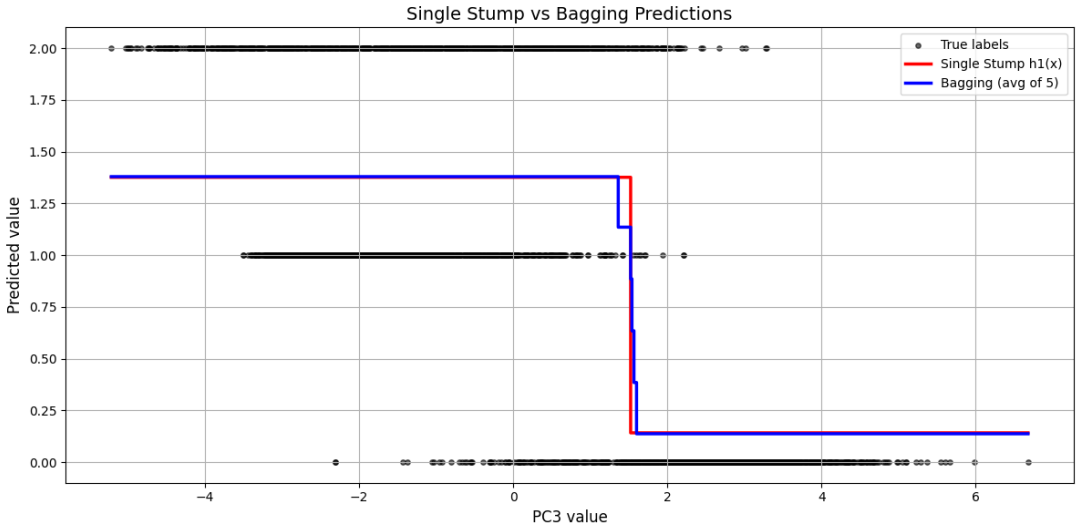
Five bootstrap datasets, each matching the training set size of 18,000, were generated by sampling with replacement. A regression decision stump was trained on each bootstrap dataset. The Out-of-Bag (OOB) Mean Squared Error was computed for each stump using the samples excluded during its respective bootstrap draw. The final bagged prediction for the test set was computed as the average of the predictions from all 5 models.

- **Average OOB Error (MSE):** 0.3514
- **Test Set MSE (Bagging):** 0.3519

Comparison: Single Stump vs. Bagging

Table 6: Single Stump vs. Bagging

Method	Test MSE	Avg OOB MSE
Single Stump	0.3580	—
Bagging ($B = 5$)	0.3519	0.3514



Analysis of the comparison plot: The plot visualizes the true class labels against the selected split feature (PC3), alongside the step-function predictions from the single stump (red line) and the bagged ensemble (blue line).

Observed differences:

- **Prediction Granularity:** The single stump produces exactly two distinct prediction values (1.3758 and 0.1413), resulting in a rigid, single-step function. This demonstrates the high bias of the base model, which forces all samples into just two rigid buckets, systematically misclassifying samples near the boundary.

- **Ensemble Smoothing:** The bagged model averages the outputs of 5 independent stumps trained on different data subsets. Because these bootstrap samples can favor slightly different optimal features or thresholds, their aggregated average yields a step function with multiple, finer transitions.

- **Bias vs. Variance Limitation:** While bagging introduces a richer set of prediction values and marginally lowers the MSE (from 0.3580 to 0.3519), the overall improvement is minor. Bagging is fundamentally a variance-reduction technique. Because a decision stump suffers from extremely high bias (a single linear split cannot capture non-linear class distributions), averaging multiple high-bias models cannot fully overcome the inherent limitations of the stump architecture.